VISVESVARAYA TECHNOLOGICAL UNIVERSITY
JNANASANGAMA, BELAGAVI - 590018

Augmented Reality and Virtual Reality (23CSE145)
Project Report On
Escape Room

Submitted in partial fulfilment for the award of degree of

Bachelor of Engineering
in
COMPUTER SCIENCE AND ENGINEERING

Submitted by
Akshath (1BG23CS010)
Chinmay Kashyap CS (1BG23CS031)

Under the Guidance of
Dr. Kavitha Jayaram
Associate Professor
BNMIT, Bengaluru

Vidyayāmruthamashnuthe

B.N.M. Institute of Technology

An Autonomous Institution under VTU

Approved by AICTE, Accredited as grade A Institution by NAAC. All eligible branches - CSE, ECE, EEE, ISE & Mech. Engg. are accredited by NBA for academic years 2018-19 to 2024-25 and valid upto 30.06.2028
URL: www.bnmit.org

Department of Computer Science and Engineering
2024 – 25

B.N.M. Institute of Technology

An Autonomous Institution under VTU

Approved by AICTE, Accredited as grade A Institution by NAAC. All eligible branches -- CSE, ECE, EEE, ISE & Mech. Engg. are accredited by NBA for academic years 2018-19 to 2024-25 and valid upto 30.06.2025

URL: www.bnmit.org

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Vidyayāmruthamashnuthe

CERTIFICATE

Certified that the Fourth Semester Introduction to AR and VR entitled "**Escape Room**" carried out by **Akshath G(1BG23CS010)** and **Chinmay Kashyap CS (1BG23CS031)** bonafide students of IV Semester B.E., **B.N.M Institute of Technology**, an Autonomous Institution under Visvesvaraya Technological University, Belagavi submitted in partial fulfillment for the Bachelor of Engineering in COMPUTER SCIENCE AND ENGINEERING, Belagavi during the year 2024-25. It is certified that all corrections/suggestions indicated for Internal Assessment have been incorporated in the report. The Introduction to AR and VR project report has been approved as it satisfies the academic requirements in respect of the Introduction to AR and VR project prescribed.

Dr. Kavitha Jayaram	**Dr. Chayadevi M L**	**Dr. S Y Kulkarni**
Associate Professor	Professor and HOD	Additional Director
Dept. of CSE	Dept. of CSE	and Principal
BNMIT, Bengaluru	BNMIT, Bengaluru	BNMIT, Bengaluru

	Name	**Signature with Date**
Examiner 1:		
Examiner 2:		

ACKNOWLEDGEMENT

We consider it a privilege to express through the pages of this report, a few words of gratitude to all those distinguished personalities who guided and inspired us in the completion of this Introduction to AR and VR project.

We would like to thank **Shri. Narayan Rao R Maanay**, Secretary, BNMEI, Bengaluru for providing an excellent academic environment in college.

We would like to thank **Prof. T J Rama Murthy**, Director, BNMIT, Bengaluru for having extended his support and encouragement during the course of work.

We would like to thank **Dr. S Y Kulkarni**, Principal and Additional Director, BNMIT, Bengaluru for his extended support and encouragement during the course of work.

We would like to express our gratitude to **Prof. Eishwar N Maanay**, Dean, BNMIT, Bengaluru for his relentless support, guidance, and encouragement.

We would like to thank **Dr. Krishnamurthy G N**, Deputy Director, BNMIT, Bengaluru for his constant encouragement.

We would like to thank **Dr. Chayadevi M L**, Professor and Head of the Department of Computer Science and Engineering, BNMIT, Bengaluru, for her support and encouragement towards the completion of the Introduction to AR and VR Project.

We would like to express our gratitude to our guide **Dr. Kavitha Jayaram**, Associate Professor of Department of Computer Science and Engineering, BNMIT, Bengaluru, who has given us all the support and guidance in completing the Introduction to AR and VR project successfully.

Akshath G
(1BG23CS010)

Chinmay Kashyap
(1BG23CS031)

ABSTRACT

This project presents the design and development of a 3D puzzle-based game titled *Escape Room*, created using the Unity game engine. The game offers an immersive, multi-level experience where players must solve a variety of puzzles to progress through four distinct rooms. Each level is designed to test different cognitive abilities — including observation, logical reasoning, spatial awareness, and memory — while gradually increasing in complexity to maintain engagement and provide a rewarding sense of progression.

Level 1 introduces the core mechanics with a simple proximity-based teleportation task, allowing players to familiarize themselves with movement and interaction. Level 2 integrates a basic mathematical puzzle requiring user input, thereby introducing logic-based gameplay. Level 3 adds a spatial challenge, where players must arrange books in a particular pattern to advance. Finally, Level 4 presents a large, intricate maze designed to test navigational skills and patience.

Throughout development, Unity's features such as C# scripting, raycasting, trigger detection, UI systems, and scene transitions were extensively utilized. The project followed an iterative development model, allowing for regular testing and refinement. The result is a cohesive and engaging escape room game that showcases practical implementation of game mechanics, level design principles, and user-centered interaction. This project serves as both a learning experience in Unity-based development and a foundation for future expansion into more complex or collaborative gameplay experiences.

TABLE OF CONTENTS

Chapter 1 – Introduction……..... 2

1.1 Background and Motivation……... 2

1.2 Problem Statement……. 3

1.3 Objectives……... 3

1.4 Scope of Project……. 4

CHAPTER 2 – Literture Survey……. 5

2.1 Related Works…… 6

2.2 Gaps Identified…….. 7

2.3 User Interaction in Puzzle-Based ……. 7

CHAPTER 3 – System Requirements…… 9

3.1 System Requirements…… 9

3.2 Hardware Requirements…… 10

3.3 Functional Requirements……... 10

CHAPTER 4 – Methodology and Implementation……... 11

4.1 Phase Description……... 12

4.2 Testing and Validation……... 12

4.3 Validation Metrics…….. 13

4.4 System Architecture……... 14

Chapter 5 – Result and discussion…….. 16

5.5 Result……... 16

5.6 Discussion……... 19

Chapter 6 – Conclusion…… 21

6.1 Conclusion…… 21

References…….. 27

LIST OF FIGURES

Figure No.	Name of Figure	Page No.
5.1	Level 1 with basic movement and interaction.	16
5.2	Displays Level 2, where the player interacts with a button to trigger a math puzzle.	17
5.3	Level 3, where the player must arrange books in a specific pattern on a shelf.	18
5.4	Level 4, a challenging 3D maze that tests the player's memory.	18
5.5	Entire game model	19
5.6	Player Movement and Collision Handling Script in Unity	20
5.7	Camera Control and Jump Mechanism Using Mouse and Keyboard	20

CHAPTER 1
INTRODUCTION

CHAPTER 1

Introduction

1.1 Background and Motivation

Escape rooms have emerged as a popular form of interactive entertainment, combining storytelling, logical puzzles, and immersive environments to challenge players' problem-solving and collaboration skills. Traditionally experienced in physical venues, escape rooms place participants in a confined space where they must complete a series of puzzles or tasks within a time limit to "escape." These experiences demand critical thinking, observation, and memory skills, often drawing from themes like mystery, adventure, or suspense.

With the rise of digital game development tools and 3D engines like Unity, escape room concepts have increasingly transitioned into virtual formats. Digital escape rooms not only broaden accessibility by eliminating the need for physical spaces but also allow for more imaginative environments, automated mechanics, and nonlinear puzzle designs. The virtual format opens up possibilities for creative gameplay, including teleportation, interactive UI puzzles, drag-and-drop mechanics, and complex mazes—all within a cohesive, user-controlled experience.

This project draws inspiration from both real-world escape rooms and digital puzzle games, aiming to create an engaging and mentally stimulating experience within a Unity-built virtual environment. The core motivation behind this work is to explore how game development principles—such as interactive design, visual feedback, and progressive difficulty—can be applied to a multi-level puzzle game that is both fun and intellectually rewarding.

1.2 Problem Statement

While digital escape room games are increasingly common, many suffer from issues related to user engagement, linear gameplay, and lack of intuitive interaction. Players often encounter static puzzles that do not evolve or offer increasing complexity

The main challenge in this project lies in designing a structured, multi-level puzzle game that introduces new types of cognitive challenges at each stage. Each level must provide a unique gameplay mechanic—from movement-based interactions to logical thinking and pattern recognition—while maintaining a consistent theme and intuitive control scheme. Additionally, the transitions between levels must feel smooth and rewarding to sustain engagement.

This project seeks to address these challenges by implementing a four-level escape room game, where each level introduces new mechanics and interactive logic using Unity's scripting and event systems. The goal is to move beyond basic static puzzles and instead offer a dynamic, hands-on experience where users actively engage with the environment and receive immediate feedback on their actions.

1.3 Objectives

The objectives of this project focus on the design, development, and testing of an interactive, multi-level escape room game built using Unity. Specific goals include:

- To design and develop an engaging 3D escape room game with four distinct levels of increasing difficulty.
- To integrate a variety of puzzle types including:
  - Level 1: Proximity-based teleportation.
  - Level 2: Math-based puzzle solving using UI input.
  - Level3: Object arrangement logic through drag-and-drop interaction.
  - Level 4: Maze navigation and memory-based spatial challenges
- To implement intuitive player movement using standard keyboard controls (WASD + Enter/Space/etc.).
- To use Unity's built-in systems (e.g., Scene-Manager, Canvas UI, ray-casting) for interaction detection and scene transitions.

1.4 Scope of Project

This project is designed to deliver a complete and functional virtual escape room experience, with focused attention on puzzle design, player interaction, and progressive level development. The scope of the project includes:

- **Game Environment Design**
  - Create a cohesive visual style across four different levels, each housed in a uniquely themed room.
  - Design puzzle elements and navigation layouts using Unity's 3D tools and asset system.

- **Player Controls and Mechanics**
  - Implement basic movement using keyboard controls.
  - Enable puzzle-specific interactions such as pressing buttons, entering values, or rearranging objects using raycasting and colliders.

- **Level-Specific Features**
  - *Level 1:* Enter a specific zone and press "Enter" to teleport to the next room.
  - *Level 2:* Approach a button, click it, and solve a math equation via an on-screen input field.
  - *Level 3:* Interact with a bookshelf to rearrange books in the correct pattern using drag-and-drop functionality.
  - *Level 4:* Navigate a large 3D maze to locate the exit and complete the game.

- **Puzzle Logic and Scene Management**
  - Use C# scripting to handle game logic, puzzle validation, and player feedback.
  - Seamlessly transition between levels using Unity's SceneManager and conditional triggers.

- **Testing and Feedback**
  - Conduct playtesting to verify usability, puzzle difficulty, and game flow.
  - Apply feedback to improve puzzle clarity, interaction responsiveness, and level balance.

CHAPTER 2
LITERATURE SURVEY

CHAPTER 2

Literature Survey

Science fiction as a genre has long explored themes of human survival, cosmic entities, and reality-bending experiences. This project draws inspiration from a variety of works across literature, film, and digital art:

2.1 Related Works

2.1.1 The Room (Mobile Game Series))

Explores a ”shimmer” zone that alters biology and physics, reflecting how unknown forces can reframe reality itself. Influenced our depiction of the 4D space inside the entity.

2.1.2 “Portal” (2007, game)

A first-person puzzle-platform game by Valve that uses spatial logic and problem-solving. It influenced the idea of progressive puzzle complexity and smooth level transitions.

2.1.3 “The Witness” (2016, game)

A visually minimalist game focused on solving line-drawing puzzles across an open world. This game inspired our approach to integrating puzzles naturally within the environment.

2.1.4 Escape Room (2019,movie)

This psychological thriller provided ideas for designing increasingly difficult escape scenarios and high-stakes environments to build suspense and player engagement

2.2 Gaps Identified

While many puzzle games focus on a single type of interaction (e.g., code-breaking, movement-based, or logic puzzles), few offer a diverse sequence of puzzle types tied together in a narrative progression. Additionally, many games lack adaptive difficulty or intuitive interaction systems that accommodate casual players. This project aims to create a cohesive multi-level escape game combining spatial puzzles, logic-based questions, pattern recognition, and navigation challenges, filling a gap in variety and progression.

2.3 User Interaction in Puzzle-Based Games

User interaction design is critical in escape room games where immersion relies on how players engage with their environment. Several interaction paradigms are used in digital puzzle games:

- **Point-and-Click Mechanics:** Common in mobile puzzle games, where players tap or click to investigate objects.
- **Drag-and-Drop Systems:** Used for object arrangement tasks, especially in logic or sequence puzzles.
- **Trigger Zones and Key-Based Inputs:** Players move through the space and interact using keyboard controls or controller inputs.

In this project, **keyboard-based interaction** was chosen to simplify input while supporting a range of actions:

- **WASD keys** for player movement,
- **Enter**, **Space**, **Ctrl**, **Z**, etc., for interacting with puzzle elements. This approach ensures accessibility, especially for users without VR or touch-based hardware, and focuses on logic-driven gameplay over sensory immersion.

CHAPTER 3

SYSTEM REQUIREMENT SPECIFICATION

CHAPTER 3

System Requirements

To successfully develop and execute the *Escape Room* game, a defined set of system requirements must be met. These requirements include the software and hardware tools needed to build the game, as well as the functional capabilities expected from the final product. This section outlines the technical prerequisites and interaction goals necessary to ensure a smooth and engaging user experience.

3.1 System Requirements

- **Operating System:** Windows 10 or Windows 11 (64-bit)
- **Game Engine:** Unity Hub with Unity 2022.3 LTS or later
- **Programming Language:** C# (for scripting in Unity)
- **3D Modeling Software:** Blender (version 2.9 or above, optional for custom asset creation)
- **Integrated Development Environment (IDE):** Visual Studio 2019/2022 or JetBrains Rider
- **Asset Repository:** Unity Asset Store (for downloading ready-to-use 3D models, UI elements, and textures)
- **Graphics and Texture Tools:** Photoshop or GIMP (for creating/editing textures and UI assets)

3.2 Hardware Requirements

- **Processor:** Intel Core i5 (8th Gen or higher) or AMD Ryzen 5 equivalent
- **Memory (RAM):** Minimum 8 GB
- **Graphics Card:** NVIDIA GeForce GTX 1050 Ti or AMD Radeon RX 560 (or better)
- **Storage:** Minimum 2 GB of free disk space for development and builds
- **Input Device:** Keyboard and mouse (required for player movement and puzzle interaction)

3.3 Functional Requirements

The *Escape Room* game must allow the player to:

General Functionalities

- Navigate the 3D environment using standard **WASD** keys.
- Rotate the camera and interact with objects using the **mouse**.
- Receive visual and/or audio feedback upon puzzle completion or level transition.

Level-Specific Interactions

- **Level 1 – Teleportation Puzzle:**
  - Detect proximity to a defined location or object.
  - Press **Enter** to teleport to the next level.
- **Level 2 – Math Puzzle:**
  - Move near a button object and press **E** or click to activate the puzzle.
  - Display a UI math question (e.g., 2 + 2 = ?) and input field.
  - Validate correct answer (**4**) to unlock and transition to Level 3.
- **Level 3 – Bookshelf Puzzle:**
  - Enable drag-and-drop of books into designated slots on the shelf.
  - Validate correct sequence or pattern for successful puzzle completion.
  - Trigger a hidden door or teleporter to progress to Level 4.
- **Level 4 – Maze Challenge:**
  - Allow free movement through a complex maze.
  - Include collision boundaries, dead ends, and visual markers.
  - Detect arrival at the maze exit to complete the game.

CHAPTER 4
METHODOLOGY AND IMPLEMENTATION

CHAPTER 4

Methodology and Implementation

The development of the *Escape Room* game followed an iterative, modular approach commonly used in game development, ensuring continuous testing and enhancement throughout each phase. The project was divided into clear stages focusing on design, interaction logic, puzzle implementation, and visual polish.

4.1 Phase Description

1. Concept Development: Defined the core game concept — a four-level escape room with increasing complexity. Identified gameplay mechanics for each level and established the visual theme for all rooms.

2. Created level maps and sketches for spatial layout. Gathered or modeled required 3D assets such as bookshelves, buttons, walls, and maze elements using Unity Asset Store and Blender

3. Wrote C# scripts to handle player interactions (e.g., button pressing, teleportation, puzzle validation). Configured colliders, triggers, and raycasting for object detection and UI responses.

4. Puzzle Implementations

- **Level 1:** Set a trigger zone and input condition to teleport the player.
- **Level 2:** Added a UI pop-up for a math equation and logic to validate the answer.
- **Level 3:** Implemented a drag-and-drop system for arranging books in the correct order.
- **Level 4:** Designed a large, complex maze and placed the exit trigger to complete the game.

5. Used Unity's SceneManager to manage transitions between levels upon puzzle completion. Added loading screens if needed.

6. Added background music, button click sounds, and puzzle completion cues. Designed minimal UI for instructions, prompts, and feedback.

7. Conducted iterative playtests to refine puzzle clarity, fix bugs, and improve user experience. Balanced level difficulty based on feedback.

4.2Testing and Validation

Game development involved functional and user-focused testing rather than traditional software validation. Each stage was tested to ensure smooth interaction, puzzle logic, and scene flow

4.1.1 Stage Testing Approach & Objectives

- **Level Logic Testing:** Ensured that each level's puzzle condition correctly triggers the transition to the next.
- **Interaction Testing:** Verified object selection, button responses, and UI inputs were functioning reliably.
- **Navigation Testing:** Checked that player movement, collision, and maze navigation were smooth and bug-free.
- **Puzzle Validation:** Tested for puzzle-solving logic — especially input validation in Level 2 and object placement in Level 3.
- **Audio & UI Testing:** Ensured that sounds matched actions and UI elements displayed as expected.
- **Cross-Platform Build Testing:** Verified game builds functioned correctly on target platforms (e.g., Windows PC).

4.3 Validation Metrics

- **Functionality Accuracy:** All scripts and puzzles execute expected results without glitches.
- **User Experience:** Players can navigate, interact, and solve puzzles intuitively.
- **Performance:** Game runs smoothly without lag or major FPS drops in any level.
- **Replayability:** Game levels provide sufficient challenge and interest to encourage multiple attempts.

4.4 System Architecture

The system architecture of the *Escape Room* game is modular, designed to separate player control, puzzle logic, and environment management. This ensures each level can operate independently while maintaining consistent interaction rules and user flow across the game.

Core Components:

- **Input Manager**
  Captures keyboard inputs (W, A, S, D, Enter, E, Space, Ctrl) and routes them to interaction handlers.

- **Player Controller**
  Manages character movement, collision detection, and camera control. Implements walking and rotation logic using Unity's Rigidbody and CharacterController components.

- **Level Manager**
  Handles the loading/unloading of each level using Unity's SceneManager. Stores progress flags and transitions between rooms based on puzzle completion.

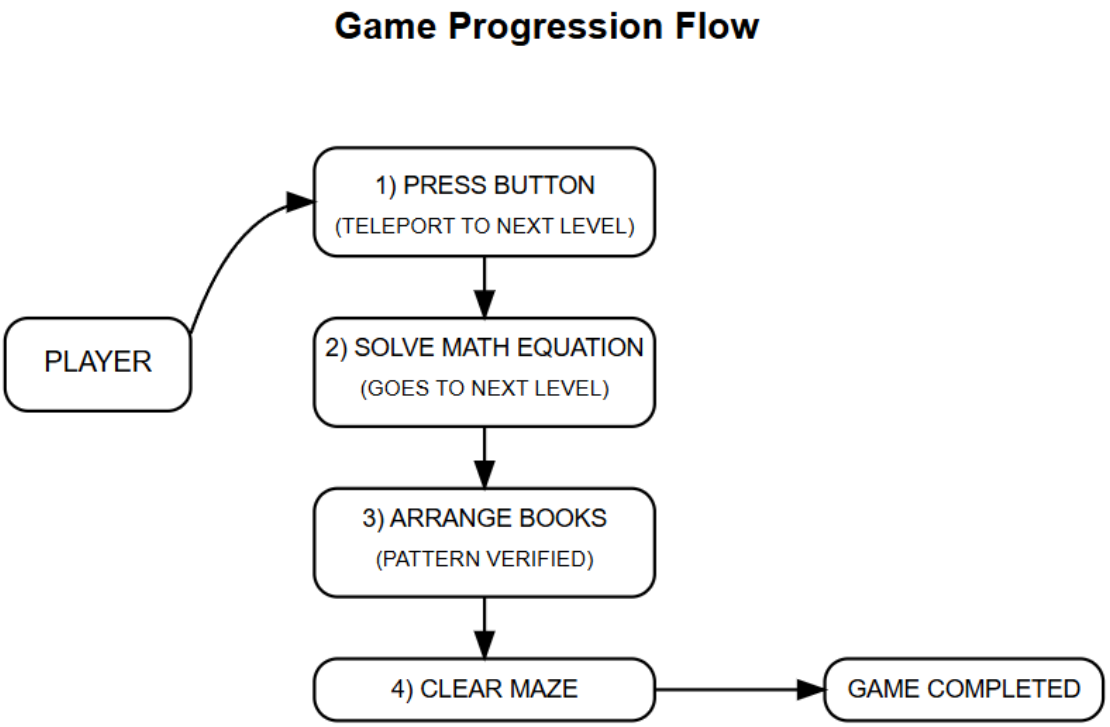
- **Puzzle Manager**
  Detects proximity to interactive objects (via trigger zones or raycasting) and manages logic for each puzzle:

  - Math puzzle (Level 2)
  - Book arrangement (Level 3)
  - Maze exit check (Level 4)

- **UI Manager**
  Displays puzzle prompts, success/failure feedback, and user input fields. Updates dynamically based on context.

System Architecture Overview (Descriptive Diagram)

Here's a **textual representation** you can convert into a flowchart or block diagram:



This layout gives a clear picture of how:

- Input feeds player control,
- Player interacts with puzzles through UI,
- Each level has its own puzzle logic module,
- SceneManager controls the transitions.

CHAPTER 5
RESULT AND DISCUSSION

CHAPTER 5

Result and Discussion

5.1 Result

The *Escape Room* game was successfully developed and implemented using Unity, consisting of four interactive and progressively challenging levels. Each level introduces a unique gameplay mechanic to test the player's cognitive, observational, and navigational skills:

- Level 1 allowed players to understand basic movement and interaction through a teleportation mechanic.
- Level 2 introduced a math-based puzzle using a UI input system, successfully validating user answers and enabling conditional progression.
- Level 3 featured an object arrangement puzzle that used drag-and-drop interaction with logical placement detection.
- Level 4 culminated in a complex 3D maze requiring player navigation, memory, and spatial awareness.

All levels were linked seamlessly using Unity's scene management system. Transitions were smooth, UI prompts were clear, and interactions were responsive. The game was tested on a Windows PC and showed no major bugs or crashes.

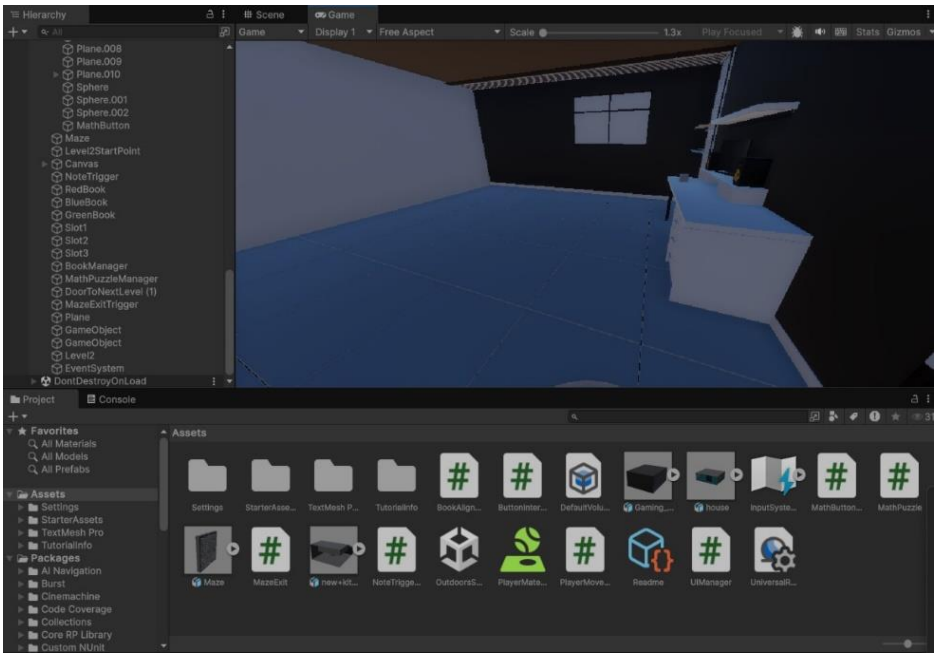


Figure 5.1 Shows the initial design of **Level 1**, where the player learns basic movement and interaction.

This image shows the initial layout of Level 1, where the player is introduced to basic controls and movement within the game environment. The player is required to walk toward a specific location. Once within the correct range, pressing the Enter key teleports them to the next level. This level sets the foundation for understanding the game's navigation and interaction system.

Figure 5.2 Displays Level 2, where the player interacts with a button to trigger a math puzzle.

In Level 2, the player encounters a room with a button placed within reach. When approached, the button can be activated, triggering a simple math puzzle displayed on the screen. The player must input the correct answer to the puzzle (e.g., solving 2 + 2 = 4) to proceed to the next level. This scene introduces logical reasoning and UI-based interaction for the first time

Figure 5.4 presents Level 4, a challenging 3D maze that tests the player's memory.

This figure depicts Level 3, where the challenge involves rearranging books on a shelf in a specific pattern. The player must drag and drop books into the correct slots based on clues or color patterns. Upon arranging the books in the correct sequence, a door or pathway unlocks, allowing progression. This level emphasizes visual observation and pattern recognition.

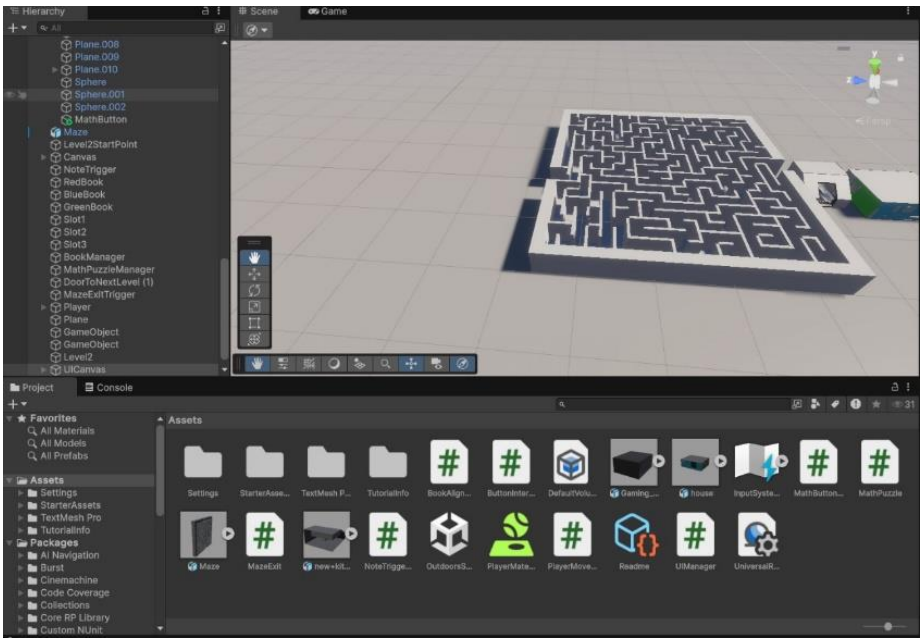


Figure 5.4 presents Level 4, a challenging 3D maze that tests the player's memory.

In the final level, the player faces a complex 3D maze filled with dead ends and misleading paths. The goal is to navigate through the maze and reach the correct exit point. This image illustrates the intricate layout of the maze, testing the player's memory, orientation, and navigation skills. Successfully reaching the exit marks the completion of the game.

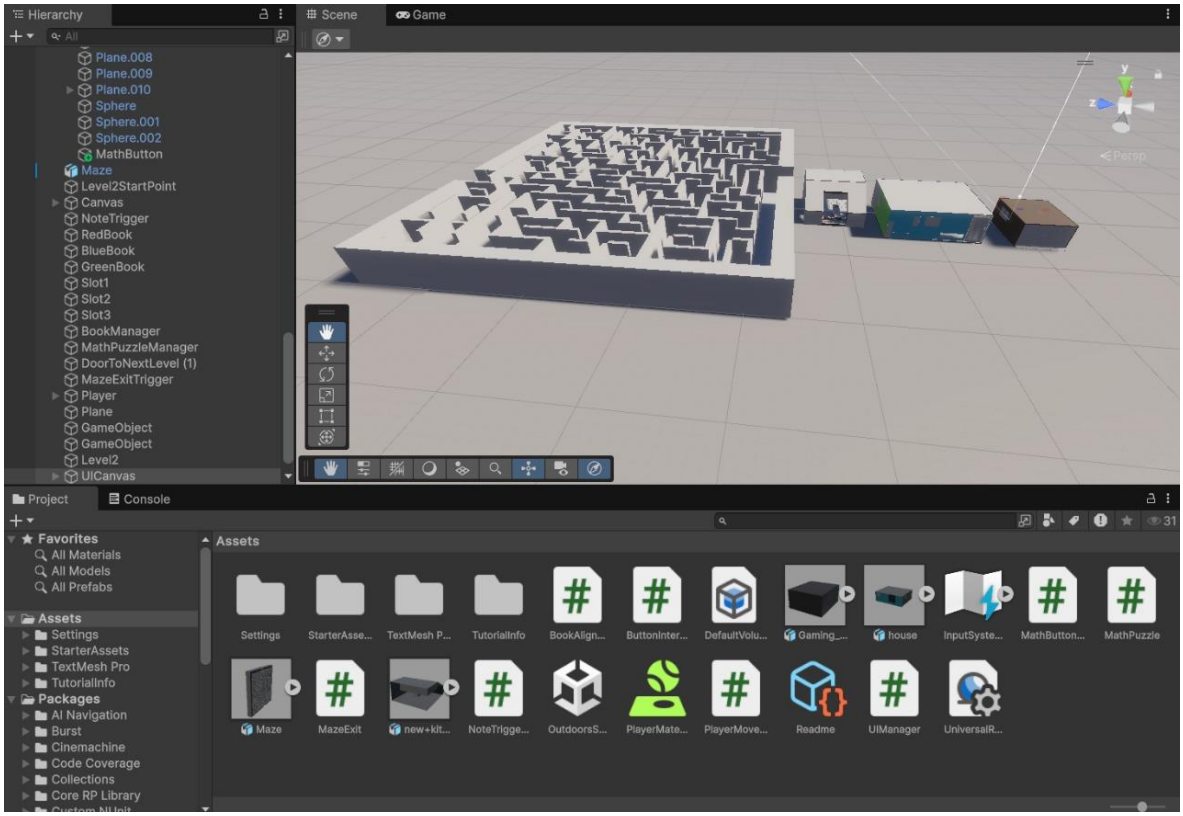


Figure 5.5 Entire game model

This image provides an overview of the complete game model, showcasing the layout of all four levels within a single unified environment. Each level is designed as a separate room or zone, with distinct visual elements and puzzle logic to differentiate them. The model reflects a modular yet connected structure, allowing smooth transitions between levels using Unity's scene or object-based teleportation system. This full-view representation helps visualize the spatial organization, progression flow, and design consistency of the escape room experience.

Code Snippets

```
    private void FixedUpdate()
    {
        float horizontal = Input.GetAxis("Horizontal");
        float vertical = Input.GetAxis("Vertical");

        Vector3 moveDirection = (transform.forward * vertical + transform.right * horizontal).normalized;
        Vector3 targetVelocity = moveDirection * moveSpeed;

        // Slide along walls
        RaycastHit hit;
        if (Physics.Raycast(transform.position, moveDirection, out hit, 0.5f))
        {
            Vector3 surfaceNormal = hit.normal;
            moveDirection = Vector3.ProjectOnPlane(moveDirection, surfaceNormal).normalized;
            targetVelocity = moveDirection * moveSpeed;
        }

        rb.linearVelocity = Vector3.Lerp(rb.linearVelocity, new Vector3(targetVelocity.x, rb.linearVelocity.y, targetVelocity.z), Time.fixedDeltaTim
        Debug.Log($"Velocity: {rb.linearVelocity}, IsGrounded: {isGrounded}");
    }

    0 references
    private void OnCollisionEnter(Collision collision)
    {
        Debug.Log($"Collided with: {collision.gameObject.name}");
    }

    0 references
    private void OnDestroy()
    {
        Cursor.lockState = CursorLockMode.None;
        Cursor.visible = true;
    }
}
```

Figure 5.6 Player Movement and Collision Handling Script in Unit

The above C# script is part of a Unity project used to control player movement through physics-based logic in the FixedUpdate() method. It captures user input from the horizontal and vertical axes and converts it into a normalized movement direction.

```
private void Update()
{
    float mouseX = Input.GetAxis("Mouse X") * mouseSensitivity * Time.deltaTime;
    float mouseY = Input.GetAxis("Mouse Y") * mouseSensitivity * Time.deltaTime;

    transform.Rotate(0f, mouseX, 0f);
    xRotation -= mouseY;
    xRotation = Mathf.Clamp(xRotation, -90f, 90f);
    playerCamera.localRotation = Quaternion.Euler(xRotation, 0f, 0f);

    // Use SphereCast for better ground detection
    isGrounded = Physics.SphereCast(transform.position, 0.3f, Vector3.down, out RaycastHit hit, groundCheckDistance + 0.1f);
    Debug.DrawRay(transform.position, Vector3.down * (groundCheckDistance + 0.1f), Color.red);

    if (Input.GetKeyDown(KeyCode.Space) && isGrounded)
    {
        rb.AddForce(Vector3.up * jumpForce, ForceMode.Impulse);
    }
}
```

Figure 5.7 – Camera Control and Jump Mechanism Using Mouse and Keyboard Input

This script handles camera rotation based on mouse input and clamps the vertical rotation for realistic head movement. It also uses SphereCast for accurate ground detection and allows the player to jump using the Space key when grounded.

5.2 Discussion

The project met its primary objective of delivering an immersive escape room experience with multiple puzzle types and increasing difficulty. The following insights were observed during development and testing:

- User Engagement: Players remained engaged through variety in challenges — from logical reasoning to spatial navigation.
- Learning Curve: Introducing simple mechanics in Level 1 helped players adapt to the control scheme before facing more complex tasks.
- Puzzle Balance: Testing helped calibrate puzzle difficulty to ensure they were neither too easy nor too obscure.

Modular Design: Each level was developed as an individual module, which improved maintainability and allowed for easy troubleshooting

Overall, the project demonstrated Unity's capabilities in building multi-level interactive games and showcased practical implementation of core game development principles such as input handling, level design, UI integration, and scene management.

CHAPTER 6
CONCLUSION

CHAPTER 6

Conclusion

The *Escape Room* project set out to design and implement a multi-level 3D puzzle game using the Unity game engine. Over the course of development, the project evolved into a well-structured and immersive experience that blends logical thinking, spatial awareness, and interaction design. Through a gradual increase in puzzle difficulty and variety in gameplay mechanics, the game successfully maintained user engagement and provided a challenging yet enjoyable experience.

The development process began with conceptualizing the game's core mechanics and visual themes. Careful planning ensured that each level introduced new gameplay elements: starting with basic movement and interaction in Level 1, progressing through input-based logical puzzles in Level 2, object arrangement tasks in Level 3, and culminating in a complex navigation challenge in the final maze level. This deliberate escalation of complexity helped to create a natural learning curve for players, allowing them to adapt and grow their problem-solving skills throughout the game.

From a technical standpoint, the project leveraged Unity’s robust development environment, including features such as C# scripting, event systems, raycasting, trigger detection, and scene management. These capabilities allowed the integration of a wide range of interactive elements, including teleportation zones, user input fields, drag-and-drop functionality, and a large-scale 3D maze. Basic UI elements were created to guide the player and provide feedback, and sound effects were incorporated to enhance the immersive quality of the game.

Overall, the *Escape Room* project accomplished its initial goals and demonstrated the potential of Unity as a platform for developing engaging and educational 3D games. It provided valuable insights into the game development pipeline, from ideation and prototyping to scripting and testing. Additionally, it showcased the importance of player-centered design, balancing challenge with accessibility.

REFERENCES

[1] Unity. *Official Unity Learn – Beginner Gameplay Scripting*. https://learn.unity.com/project/beginner-gameplay-scripting

[2] Brackeys. *How to Make a Game in Unity (Beginner Tutorial)*. YouTube, 2020. https://www.youtube.com/watch?v=IlKaB1etrik

[3] Unity. *Scene Management in Unity – Loading Levels and Game States*. https://docs.unity3d.com/Manual/MultiSceneEditing.html

[4] Code Monkey. *How to Make Interactable Objects in Unity*. YouTube, 2021. https://www.youtube.com/watch?v=V1y-mbWM3B8

[5] GameDevTV. *Puzzle Game Design Principles*. https://www.gamedev.tv/p/puzzle-course

[6] Unity Asset Store. *Free 3D Models and Game Assets*. https://assetstore.unity.com

[7] Dani. *I Made a Game with Unity in 1 Week – Escape Room Challenge*. YouTube, 2019. https://www.youtube.com/watch?v=GZb6LNwDvJI

[8] Sykoo. *Level Design in Unity – Modular Game Environments*. YouTube, 2020. https://www.youtube.com/watch?v=fdDrkG_AuVw

[9] Unity. *Raycasting Explained – Detecting Object Interactions*. https://docs.unity3d.com/ScriptReference/Physics.Raycast.html

[10] Mix and Jam. *Recreating Game Mechanics – God of War Puzzle Style in Unity*. YouTube, 2020. https://www.youtube.com/watch?v=QpN1g6h1F2Y